Mappa Concettuale Completa — Financial Project Performance Analysis

Contesto Generale

Il progetto analizza 3 titoli finanziari **quotati contemporaneamente su più borse**: - **UL** (Unilever) → presente su NYSE (reale), LSE e AMS (simulate) - **SHELL** → identico a UL - **HSBC** → presente su NYSE come ADR (reale), LSE (simulata), + proxy sentimento Hong Kong

L'obiettivo è: **rilevare finestre di arbitraggio** — momenti in cui lo stesso titolo ha prezzi diversi su borse diverse, abbastanza diversi da coprire i costi di transazione e il rischio.

Fase 0 — Configurazione (src/config_simbols.py)

Cosa fa concretamente: Definisce tutte le costanti usate da tutti gli altri moduli. Nessun calcolo, solo strutture dati.

SYMBOL_CONFIG (righe 10-38): dizionario con 3 chiavi (UL, SHELL, HSBC). Per ogni simbolo specifica: - `base_ccy`: la valuta del mercato reale → sempre USD - `base_venue`: US o US_ADR (HSBC è un American Depositary Receipt) - `simulate_venues`: quali borse simulare e a che tasso di cambio verso EUR - LSE → GBP, con fx_to_eur = 1.17 (1 GBP = 1.17 EUR) - AMS → EUR, con fx_to_eur = 1.00 (già in euro) - HSBC ha un flag aggiuntivo: `use_hk_close_sentiment_proxy: True`

SIMULATION_CONFIG (righe 40-72): 46 parametri. I più importanti:

Parametro	Valore	Significato
`sync_tolerance_ms`	10	Massimo scarto temporale tra due quote di venue diverse per essere considerate "contemporanee"
`price_noise_bps_std`	0.6	Deviazione standard del rumore di prezzo in basis points (1 bps = 0.01%)
`spread_multiplier_mean`	1.05	Lo spread simulato è in media il 5% più largo di quello reale
`arb_window_prob`	0.00015	Probabilità che ogni singolo tick generi una finestra di arbitraggio artificiale
`arb_window_bps_min/max`	1.5 / 5.0	Ampiezza minima/massima dello shock di arbitraggio

Parametro	Valore	Significato
`arb_window_ms_min/max`	1 / 6	Durata minima/massima dello shock in millisecondi
`risk_lambda`	6.0	Peso del rischio nella soglia dinamica
`tau_ms_for_risk`	10	Tempo di detenzione ipotetico in ms (usato nel calcolo del rischio)
`cost_unit_eur`	0.0020	Costo di transazione per unità in EUR
`fixed_buffer_eur`	0.0015	Costo fisso aggiuntivo per ogni trade
`min_duration_ms`	1.0	Una finestra di arb deve durare almeno 1ms
`min_obs_per_window`	2	Una finestra di arb deve avere almeno 2 osservazioni
`trade_block_size`	1000	Dimensione blocco di simulazione in EUR
`trade_step_ms`	10	Passo temporale per la simulazione esecuzione

Alla riga 8, all'import del modulo, crea fisicamente la cartella `data/processed/simulated/` se non esiste.

Fase 1 — Preprocessing Dati Reali

src/preprocessing_mbp1.py — Order Book Level 1

Cosa sono i dati di input: File in formato `.mbp-1.dbn.zst` (Databento Binary Notation, compressi con zstd). Contengono snapshots del miglior bid e ask dell'order book a ogni modifica di prezzo.

Percorso input: `data/raw/databento/{simbolo}/mbp1/`

Step 1 — `load_single_dbn()` (riga 11): Apre un singolo file `.dbn.zst` con la libreria Databento, lo converte in DataFrame pandas. Se il timestamp `ts_event` è usato come indice del DataFrame, lo riporta a colonna normale con `reset_index()`.

Step 2 — `load_all_mbp1_files()` (riga 21): Trova tutti i file `*.mbp-1.dbn.zst` nella cartella con `sorted(raw_dir.glob(...))`, li carica uno per uno e li concatena in un unico DataFrame con `pd.concat()`.

Step 3 — `clean_mbp1()` (riga 35): - Verifica che esistano le 5 colonne obbligatorie: `ts_event, bid_px_00, ask_px_00, bid_sz_00, ask_sz_00` - Converte `ts_event` in datetime UTC con `pd.to_datetime(..., utc=True)` - Converte prezzi e size in numerico con `pd.to_numeric(..., errors="coerce")` — i valori non convertibili diventano NaN - Elimina tutte le righe che hanno almeno un NaN nelle 5 colonne obbligatorie - Ordina per timestamp crescente - Aggiunge 5 colonne calcolate: - `symbol`: nome del titolo (stringa costante, es. "UL") - `spread = ask_px_00 - bid_px_00`

→ costo implicito di comprare e rivendere subito - `mid = (bid + ask) / 2` → prezzo medio tra bid e ask - `event_date = ts_event.dt.date` → solo la data, senza ora - `ts_event_ms = ts_event.astype("int64") // 10^6` → timestamp in millisecondi come intero (utile per join veloci) - Seleziona e riordina solo le 10 colonne finali

Step 4 — `process_symbol()` (riga 75): Orchestratore. Costruisce il percorso della cartella raw, chiama load + clean, salva il risultato in `data/processed/{symbol}_mbp1.parquet`.

`main()` (riga 100): Chiama `process_symbol` tre volte — nota che Shell ha la cartella con la S maiuscola: `process_symbol("SHELL", "Shell")`.

Output: `data/processed/UL_mbp1.parquet`, `SHELL_mbp1.parquet`, `HSBC_mbp1.parquet`

---

src/preprocessing_trades.py — Trade Tick

Struttura identica a preprocessing_mbp1.py ma per i file `*.trades.dbn.zst`.

La differenza sta in `clean_trades()`: le colonne obbligatorie sono `ts_event, price, size` (invece di bid/ask/sz). La colonna `side` (Buy/Sell) viene inclusa **solo se presente** nel file raw.

Output: `data/processed/UL_trades.parquet`, `SHELL_trades.parquet`, `HSBC_trades.parquet`

Fase 2 — Calendario degli Overlap (src/overlap_calendar.py)

Problema da risolvere: Cercare arbitraggi ha senso **solo quando tutte le borse coinvolte sono aperte simultaneamente**. Questo modulo calcola esattamente quegli intervalli.

Step 1 — `load_reference_dates()` (riga 27): Legge `UL_trades.parquet` e ne estrae le date uniche. Queste date definiscono il range del progetto (solo i giorni per cui abbiamo dati reali). Se esiste la colonna `event_date` la usa direttamente, altrimenti converte `ts_event` in timezone Europe/Rome e normalizza a mezzanotte.

Step 2 — `to_rome_timestamp()` (riga 51): Funzione di utilità. Dato un giorno e un orario (es. 09:30 ora di New York), costruisce il timestamp nella timezone del mercato e lo converte in Europe/Rome. Tutti i tempi sono standardizzati a Roma perché l'autore del progetto è italiano e vuole un riferimento locale coerente.

Step 3 — `build_market_session_bounds()` (riga 59): Per un singolo giorno, calcola apertura e chiusura dei 3 mercati **espressa in ora di Roma**: - LSE: apre alle 08:00 London → tipicamente 09:00 Roma (10:00 in ora legale) - AMS: apre alle 09:00 Amsterdam → identico all'ora di Roma - NYSE: apre alle 09:30 New York → 15:30 Roma (14:30 GMT)

Step 4 — `compute_overlap()` (riga 82): Data una lista di orari di apertura e una di chiusura, l'overlap è `max(aperture) → min(chiusure)`. Se `max_apertura > min_chiusura` l'overlap è 0 (nessuna sovrapposizione).

Step 5 — `build_overlap_rows()` (riga 90): Itera su tutte le date, scarta sabato/domenica (`dayofweek >= 5`), e per ogni giorno feriale crea 3 righe: - **UL + triple**: overlap di AMS + LSE + US (tutti e 3 devono essere aperti) - **SHELL + triple**: identico a UL (stesso calcolo, stessi mercati) - **HSBC + pairwise**: overlap solo di LSE + US (non c'è AMS per HSBC)

Scarta le righe con `overlap_seconds == 0`.

`main()` (riga 205): Chiama le funzioni nell'ordine, salva il risultato.

Output: `data/processed/market_overlaps.parquet`

Schema: `symbol, overlap_type, date_rome, start_rome, end_rome, overlap_seconds, overlap_hours, markets`

Esempio concreto: Per UL in un giorno feriale estivo (ora legale), l'overlap triplo inizia alle 15:30 (NYSE apre, LSE e AMS già aperte) e finisce alle 17:30 (AMS chiude) — circa 2 ore di overlap.

Fase 3 — Filtro per Overlap (src/filter_by_overlap.py)

Scopo: Ridurre il volume di dati tenendo solo i tick avvenuti durante gli overlap. Se si cerca arbitraggio, i tick fuori dall'overlap sono irrilevanti (la controparte non è disponibile).

`load_data()` (riga 16): Carica 3 parquet: MBP, trades, e market_overlaps.

`filter_by_overlap()` (riga 25): 1. Converte `ts_event` in UTC datetime 2. Crea colonna `date_rome`: converte ogni timestamp in Europe/Rome e prende solo la data 3. Fa un **inner join** con la tabella degli overlap su `date_rome` — questo aggiunge le colonne `start_rome` ed `end_rome` per ogni riga 4. Applica la maschera temporale: `start_rome <= ts_event <= end_rome` 5. Ritorna solo le righe che soddisfano entrambe le condizioni

`process_symbol()` (riga 47): Filtra gli overlap per simbolo e tipo (`triple` per UL/SHELL, `pairwise` per HSBC), poi applica il filtro sia agli MBP sia ai trades.

Output: - `data/processed/UL_mbp1_overlap.parquet` (subset dei tick durante overlap) - `data/processed/UL_trades_overlap.parquet` - Analogamente per SHELL e HSBC

Fase 4 — Simulazione Venue Mancanti (src/simulate_missing_markets.py)

Perché serve: Non abbiamo dati reali per LSE e AMS. Li generiamo sinteticamente partendo dai prezzi US, aggiungendo realismo (rumore, spread più largo, variazioni di liquidità) e **iniettando artificialmente piccole finestre di arbitraggio**.

`_load_base()` (riga 9): Ha una logica di priorità — preferisce i file `*_overlap.parquet` (già filtrati) ai file completi. Questo assicura che la simulazione avvenga solo sui tick rilevanti.

`_generate_persistent_windows()` (riga 75) — la funzione più critica del modulo: 1. Crea un array numpy di zeri lungo quanto il DataFrame 2. Pre-campiona in un solo passaggio tutti i tick "seme" con `np.where(rng.random(n) < 0.00015)` — molto più efficiente di un loop 3. Per ogni seed: - Genera uno shock casuale in bps: tra 1.5 e 5.0 bps, segno casuale (+ o -) - Genera una durata casuale: tra 1 e 6ms - Trova l'indice di fine con `np.searchsorted` (ricerca binaria, O(log n)) - Aggiunge lo shock a tutti i tick tra idx e end_idx 4. Ritorna la serie degli effetti

Questo crea shock di prezzo **persistenti e realistici**: non rumore bianco, ma deviazioni che durano qualche millisecondo, esattamente come succede nei mercati reali quando l'informazione si propaga con ritardo tra venue.

`_simulate_mbp_for_venue()` (riga 102) — costruisce il libro ordini sintetico: 1. Converte bid/ask base da USD a EUR: `bid_eur = bid_usd * 0.92` 2. Calcola mid e spread in EUR 3. Genera **4 array di rumore casuali** con il generatore numpy `rng` : - `micro_noise` : distribuzione normale, μ=0, σ=0.6/10000 → rumore di circa 0.006% sul prezzo - `spread_mult` : distribuzione normale, μ=1.05, σ=0.03, clippata a [0.7, 1.6] → lo spread europeo è mediamente il 5% più largo - `size_mult_bid` e `size_mult_ask` : distribuzioni indipendenti, μ=0.95, σ=0.10, clippati a [0.2, 2.5] → la profondità del book varia ±25% in media 4. Chiama `_generate_persistent_windows()` per ottenere l'effetto arb 5. Calcola il mid sintetico: `syn_mid = base_mid_eur * (1 + micro_noise + arb_effect)` 6. Calcola spread sintetico: `syn_spread = base_spread_eur * spread_mult` 7. Ricostruisce bid e ask: `bid = syn_mid - spread/2` , `ask = syn_mid + spread/2` 8. Riconverte da EUR alla valuta locale della venue: `bid_gbp = bid_eur / 1.17` 9. Arrotonda le size a interi con `np.round()` , garantisce minimo 1 con `np.maximum(1, ...)` 10. Aggiunge colonne metadata: `venue, ccy, fx_to_eur, bid_px_eur, ask_px_eur, simulated=True`

`_simulate_trades_for_venue()` (riga 179): Più semplice. Applica solo micro-noise ai prezzi e scala le size. Non inietta finestre di arb (quelle sono nel book, non nei trade).

`_build_hk_close_sentiment_proxy()` (riga 217) — solo per HSBC: 1. Calcola il mid: `(bid + ask) / 2` 2. Ricampiona a frequenza giornaliera con `.resample("1D").last()` → prende l'ultimo mid del giorno come "prezzo di chiusura" 3. Calcola i rendimenti percentuali giornalieri: `pct_change()` 4. Applica una media mobile esponenziale con span=5 (`ewm(span=5)`): smussamento che dà peso decrescente ai giorni precedenti 5. Il risultato è una proxy del sentimento di Hong Kong: se HSBC è salita molto a HK negli ultimi 5 giorni, questo valore è positivo

`simulate_missing_markets(seed=42)` (riga 239): Orchestratore. Il seed fisso (42) garantisce **riproducibilità**: eseguendo il codice più volte si ottengono sempre gli stessi dati sintetici.

Output per simbolo: - `data/processed/simulated/UL_LSE_mbp1_sim.parquet` - `data/processed/simulated/UL_LSE_trades_sim.parquet` - `data/processed/simulated/UL_AMS_mbp1_sim.parquet` - `data/processed/simulated/UL_AMS_trades_sim.parquet` - `data/processed/simulated/HSBC_LSE_mbp1_sim.parquet` -

`data/processed/simulated/HSBC_LSE_trades_sim.parquet` - `data/processed/simulated/HSBC_hk_close_sentiment_proxy.parquet`

Fase 5 — Preparazione Cross-Market (src/cross_market_prepare.py)

Scopo: Creare un'unica tabella per simbolo che contenga le quote di **tutte le venue** (reale + simulate) con un'etichetta `market_key` che le distingue.

`_load_real_mbp()` (riga 12): Legge il parquet del mercato base (dati reali US), aggiunge `venue="BASE_REAL"` e `simulated=False`.

`_load_sim_mbp()` (riga 21): Legge il parquet simulato già prodotto nella fase 4.

`prepare_cross_market_inputs()` (riga 28): 1. Per ogni simbolo, carica il reale e tutti i simulati 2. Dal reale estrae solo 7 colonne: `ts_event, bid_px_00, ask_px_00, bid_sz_00, ask_sz_00, venue, simulated`, e aggiunge `market_key = "BASE_REAL"` 3. Dal simulato estrae 9 colonne (le stesse + `bid_px_eur, ask_px_eur`), e aggiunge `market_key = nome_venue` (es. "LSE") 4. Concatena tutti i frame verticalmente con `pd.concat(axis=0)` — ogni tick viene "etichettato" con la sua venue 5. Ordina per `ts_event` poi `market_key` — le quote contemporanee di venue diverse si trovano vicine 6. Salva in `data/processed/cross_market/{symbol}_cross_market_mbp1.parquet`

Risultato pratico: Ogni riga della tabella output ha un timestamp e dice "a questo istante, su questa venue, il bid era X e l'ask era Y". Le tre venue (BASE_REAL, LSE, AMS) compaiono in sequenza per ogni momento di mercato.

Fase 6 — Rilevamento Arbitraggio (src/cross_market_engine.py)

Il cuore analitico del progetto. Ogni funzione ha un ruolo preciso.

`_load_cross_market()` (riga 18): Legge il parquet cross-market, assicura che `ts_event` sia UTC, ordina per `market_key + ts_event`.

`_load_overlaps()` (riga 30): Legge `market_overlaps.parquet`, converte i timestamp da Europe/Rome a UTC (perché tutti i dati di mercato sono in UTC). Filtra per simbolo e tipo di overlap. Stampa statistiche: numero di giorni, ore medie/min/max di overlap.

`_apply_overlap_filter()` (riga 65): Identica a `filter_by_overlap.py` ma applicata ai dati cross-market. Fa join su `date_rome` e applica la maschera temporale. Riduce drasticamente il numero di righe.

`_ensure_eur_quotes()` (riga 89): Le venue simulate hanno già `bid_px_eur` e `ask_px_eur`. BASE_REAL no (ha prezzi in USD). Questa funzione calcola i prezzi EUR per BASE_REAL: `bid_eur = bid_usd * 0.92`. Aggiunge anche `mid_px_eur` e `spread_eur`.

`_get_pairs_for_symbol()` (riga 111): Definisce le coppie da analizzare: - UL, SHELL: 3 coppie → `(BASE_REAL, LSE)`, `(BASE_REAL, AMS)`, `(LSE, AMS)` - HSBC: 1 coppia → `(BASE_REAL, LSE)`

`_rename_for_merge()` (riga 122): Rinomina le colonne per prepararle al merge. Es. per mercato A="BASE_REAL": `bid_px_eur` → `base_real__bid_px_eur`. Per mercato B="LSE": `bid_px_eur` → `lse__bid_px_eur`. Questo permette di avere entrambi i mercati sulla stessa riga dopo il join.

`_synchronize_pair()` (riga 139) — il punto più delicato:

- Separa i dati delle due venue
- Rinomina le colonne di entrambe
- Usa `pd.merge_asof` con `direction="backward"` e `tolerance=10ms`

Cosa fa `merge_asof` backward? Per ogni tick del mercato A al tempo T, trova il tick più recente del mercato B con timestamp ≤ T e ≤ T + 10ms. **Questo simula la realtà**: quando vedi un prezzo su NYSE, sai qual era l'ultimo prezzo su LSE entro i 10ms precedenti.

- Scarta le righe dove i prezzi EUR mancano (NaN)

`_compute_pair_metrics()` (riga 173): - `gap_a_sell_b_buy = bid_A - ask_B`: se positivo, puoi vendere ad A e comprare a B con profitto lordo - `gap_b_sell_a_buy = bid_B - ask_A`: direzione inversa - `qty_a_sell_b_buy = min(bid_sz_A, ask_sz_B)`: la quantità massima eseguibile è limitata dal lato più sottile del book - `gap_diff`: variazione del gap tick per tick (derivata) - `sigma` (rolling std, finestra 200 osservazioni, min 20): volatilità locale del gap — misura quanto è instabile l'opportunità

`_dynamic_threshold()` (riga 195) — la formula chiave:

```
threshold = cost_unit + fixed_buffer + risk_lambda × σ × √(tau_sec)
          = 0.0020    + 0.0015       + 6.0 × σ × √(0.010)
```

- `cost_unit = 0.0020 EUR`: costo fisso per unità (commissioni, market impact)
- `fixed_buffer = 0.0015 EUR`: margine di sicurezza fisso
- `risk_lambda × σ × √(tau_sec)`: componente di rischio — se il gap è volatile (σ alto), la soglia sale. Con τ=10ms e λ=6, questa componente è `6 × σ × 0.1 = 0.6σ`

Il risultato: la soglia si **adatta** ai periodi di alta volatilità. Un gap di 0.004 EUR potrebbe essere sufficiente quando il mercato è calmo (σ piccola) ma insufficiente quando è agitato.

`_extract_windows_and_samples()` (riga 208): 1. Calcola `threshold_eur` per ogni tick 2. Marca ogni tick come `is_open = gap > threshold` 3. Identifica i "blocchi contigui" di tick dove `is_open=True`: ogni blocco è una finestra di arbitraggio 4. Assegna `window_id` incrementale a ogni blocco con `cumsum(block_start)` 5. Filtra solo i tick aperti 6. Aggrega per `window_id`: statistiche di t_open, t_close, durata, gap (min/mean/max), soglia, qty, sigma 7. Applica filtri minimi: `duration_ms >= 1.0` e `n_obs >= 2` 8. Ritorna sia la tabella aggregata (una riga = una finestra) sia i campioni grezzi (una riga = un tick dentro una finestra)

`_process_symbol()` (riga 264): Orchestratore per un simbolo. Stampa quante righe si perdono con il filtro overlap. Per ogni coppia di venue analizza entrambe le direzioni (vendi A compra B, e vendi B compra A).

`build_opportunity_tables()` (riga 322): Salva i risultati.

Output:

`data/processed/opportunities/{symbol}_opportunities.parquet` — una riga per finestra di arbitraggio

Schema: `window_id, symbol, pair, direction, t_open, t_close, duration_ms, n_obs, threshold_open/mean, min/mean/max_gap_eur, book_qty_open/mean, sigma_open/mean`

`data/processed/opportunity_samples/{symbol}_opportunity_samples.parquet` — una riga per tick dentro le finestre valide

Schema: `ts_event, gap_eur, book_qty, sigma_local, threshold_eur, is_open, window_id, symbol, pair, direction`

Fase 7 — Simulazione Trade (src/trade_simulation.py)

Scopo: Data una finestra di arbitraggio, simulare cosa succederebbe se si eseguisse un trade ogni 10ms, e calcolare il P&L netto.

`_load_samples()` (riga 15): Carica i campioni granulari (output fase 6), ordina per window_id + ts_event.

`_resample_window_every_10ms()` (riga 29): 1. Crea una griglia temporale regolare ogni `trade_step_ms=10ms` da `t_open` a `t_close` 2. Usa `merge_asof backward` per abbinare ogni punto della griglia all'ultimo campione osservato 3. Scarta le righe dove `gap_eur` o `threshold_eur` sono NaN 4. Risultato: serie regolare di snapshot ogni 10ms all'interno della finestra

`_risk_buffer()` (riga 57): Calcola la componente di rischio: `risk_lambda × σ × √(tau_sec) = 6.0 × σ × √(0.010) = 0.6σ`. Stessa formula della soglia dinamica, isolata come funzione.

`simulate_trading()` (riga 65) — loop principale:

Per ogni window_id: 1. Ricampiona a 10ms 2. Per ogni punto della griglia: - `gross_profit_eur = 1000 × gap_eur` (trade_block_size × gap per unità) - `net_profit_eur = 1000 × gap_eur - 1000 × cost_unit - 1000 × risk_buffer - fixed_buffer` - In formula: `net = 1000 × (gap - 0.002 - 0.6σ) - 0.0015` 3. Aggrega per window: somma tutti i blocchi → P&L totale della finestra

Stampa: numero finestre, blocchi totali, profitto lordo e netto in EUR.

Output:

`data/processed/simulated_trades/{symbol}_trade_blocks.parquet` — ogni blocco a 10ms con P&L

Schema: `window_id, symbol, pair, direction, ts_event, trade_block_size, gap_eur, threshold_eur, sigma_local, book_qty, cost_unit_eur, fixed_buffer_eur, risk_buffer_unit_eur, gross_profit_eur, net_profit_eur`

`data/processed/simulated_trades/{symbol}_trade_summary.parquet` — aggregato per finestra

Schema: `window_id, symbol, pair, direction, n_blocks, total_qty, gross_profit_total_eur, net_profit_total_eur, avg_gap_eur, max_gap_eur, avg_sigma_local`

Fase 8 — Event Log per Process Mining (src/event_log_builder.py)

Scopo: Trasformare i dati di mercato in un formato leggibile da strumenti di **process mining** (PM4Py, Disco, ProM). Il process mining scopre automaticamente il "processo" ricorrente nascosto nei dati.

Nota: Questo modulo lavora sui dati raw (non filtrati per overlap) e si concentra sul mercato US (09:30-16:00 NY).

`filter_regular_market_hours()` (riga 44): - Converte `ts_event` in timezone America/New_York - Calcola i minuti dall'inizio del giorno: `ora*60 + minuto` - Tieni solo: `dayofweek < 5` (lunedì-venerdì) AND `570 <= minuti < 960` (09:30-16:00)

`build_mbp_events()` (riga 60): Itera riga per riga (loop Python) confrontando ogni tick con il precedente: - Se `bid != prev_bid` → evento `bid_change` - Se `ask != prev_ask` → evento `ask_change` - Se `spread > prev_spread` → evento `spread_widen` - Se `spread < prev_spread` → evento `spread_narrow`

Ogni evento ha: `timestamp, activity, symbol`

`build_trade_events()` (riga 130): Più semplice. Ogni riga del parquet trades diventa un evento `trade`.

`assign_cases_by_event()` (riga 146) — la logica di process mining: - Ordina tutti gli eventi per timestamp - Il primo evento apre il caso 1 - Ogni volta che l'attività precedente era `trade`, il prossimo evento apre un **nuovo caso** - Assegna `case_id` incrementale con `cumsum(new_case)`

Perché questa logica? Un "caso" rappresenta un ciclo del mercato: tutto ciò che accade **tra un trade e il successivo** (aggiornamenti del book, variazioni di spread) forma un'istanza del processo. Studiare tanti casi permette di scoprire pattern: "tipicamente dopo uno spread_widen segue un bid_change, poi un trade".

`main()` (riga 190): Accetta il simbolo da CLI (`python src/event_log_builder.py SHELL`), default UL.

Output: `data/processed/event_logs/{symbol}_event_log_event_driven_trade.csv`

Schema: `case_id, activity, timestamp`

(Formato standard per tool di process discovery)

Flusso Completo delle Dipendenze

```
data/raw/databento/
    ├─ UL/mbp1/*.mbp-1.dbn.zst
    ├─ Shell/mbp1/*.mbp-1.dbn.zst        (nota: cartella "Shell" non "SHELL")
    └─ HSBC/mbp1/*.mbp-1.dbn.zst
           │
           ▼ preprocessing_mbp1.py
    {symbol}_mbp1.parquet  (bid, ask, spread, mid, ts_event_ms)

data/raw/databento/{symbol}/trades/*.trades.dbn.zst
           │
           ▼ preprocessing_trades.py
    {symbol}_trades.parquet  (price, size, side)

    UL_trades.parquet ──► overlap_calendar.py ──► market_overlaps.parquet
    (usato solo per estrarre le date di riferimento del progetto)

    {symbol}_mbp1.parquet ──┐
    {symbol}_trades.parquet ─┼─► filter_by_overlap.py ──► {symbol}_mbp1_overlap.parquet
    market_overlaps.parquet ─┘                          └─► {symbol}_trades_overlap.parquet

    {symbol}_mbp1_overlap.parquet ──► simulate_missing_markets.py
    {symbol}_trades_overlap.parquet        │
    config_simbols.py (parametri)          ├─► {symbol}_LSE_mbp1_sim.parquet
                                           ├─► {symbol}_LSE_trades_sim.parquet
                                           ├─► {symbol}_AMS_mbp1_sim.parquet  (non HSBC)
                                           ├─► {symbol}_AMS_trades_sim.parquet (non HSBC)
                                           └─► HSBC_hk_close_sentiment_proxy.parquet

    {symbol}_mbp1.parquet (reale BASE_REAL) ──┐
    {symbol}_{venue}_mbp1_sim.parquet ────────┼─► cross_market_prepare.py
                                              └─► {symbol}_cross_market_mbp1.parquet
                                                  (tutte le venue in un'unica tabella con
market_key)

    {symbol}_cross_market_mbp1.parquet ──┐
    market_overlaps.parquet ─────────────┼─► cross_market_engine.py
    config_simbols.py (soglie) ──────────┤       │
                                         │       ├─► {symbol}_opportunities.parquet
                                         │       │   (una riga = una finestra arb)
                                         │       └─► {symbol}_opportunity_samples.parquet
                                         │           (una riga = un tick dentro la finestra)

    {symbol}_opportunity_samples.parquet ──► trade_simulation.py
    config_simbols.py (costi) ─────────────┤       │
                                           │       ├─► {symbol}_trade_blocks.parquet
                                           │       │   (P&L ogni 10ms)
                                           │       └─► {symbol}_trade_summary.parquet
                                           │           (P&L aggregato per finestra)

    {symbol}_mbp1.parquet ──┐
    {symbol}_trades.parquet ─┼─► event_log_builder.py ──► event_log_{symbol}_event_driven_trade.csv
                            │   (separato, usa dati raw)     (input per PM4Py/Disco)
```

Cosa Produce il Progetto — Output Finali

File	Contenuto	Usato per
`opportunities/{sym}_opportunities.parquet`	Elenco finestre arb con statistiche aggregate	Analisi delle opportunità trovate
`opportunity_samples/{sym}_opportunity_samples.parquet`	Dati tick-by-tick dentro le finestre	Input per trade simulation
`simulated_trades/{sym}_trade_blocks.parquet`	P&L lordo/netto per ogni blocco di esecuzione	Analisi di profittabilità granulare
`simulated_trades/{sym}_trade_summary.parquet`	P&L totale per finestra di arbitraggio	Confronto tra opportunità
`event_logs/{sym}_event_log_*.csv`	Log di processo formato case/activity/timestamp	Process mining (PM4Py, Disco, ProM)
`simulation_validation/simulation_validation_summary.csv`	Confronto statistico reale vs simulato	Verifica qualità simulazione